

Lecture No. 3

Reading Material

Deitel & Deitel – C++ How to Program

chapter 1
1.19, 1.20, 1.21,

1.22

Summary

First C program

Variables

Data Types

Arithmetic Operators

Precedence of Operators

Tips

First C program

The best way to learn C is to start coding right away. So here is our very first program in C.

Right program.

```
#include <iostream>
Using namespace std;
Int main()
{
    Cout <<"virtual university of pakistan";
    Return 0;
}
```

#include: This is a pre-processor directive. It is not part of our program; it is an instruction to the compiler. It tells the C compiler to **include** the contents of a file, in this case the system file **iostream.h**. The compiler knows that it is a system file, and therefore looks for it in a special place. The features of preprocessor will be discussed later. For the time being take this line on faith. You have to write this line. The sign # is known as HASH and also called SHARP.

<iostream.h>

This is the name of the library definition file for all Input Output Streams. Your program will almost certainly want to send stuff to the screen and read things from the keyboard. **iostream.h** is the name of the file in which has code to do that work for you

main()

The name **main** is special, in that the **main** is actually the one which is run when your program is used. A C program is made up of a large number of functions. Each of these is given a name by the programmer and they refer to each other as the program runs. C regards the name "**main**" as a special case and will run this function first. If you forget to have a main function, or mistype the name, the compiler will give you an error.

Notice that there are parentheses ("()"), normal brackets) with main. Here the parentheses contain nothing. There may be something written inside the parentheses. It will be discussed in next lectures.

```
{ }
```

Next, there is a curly bracket also called braces("{ }"). For every open brace there must be a matching close. Braces allows to group together pieces of a program. The body of main is enclosed in braces. Braces are very important in C; they enclose the blocks of the program.

```
cout << " Welcome to Virtual University of Pakistan"
cin>>
```

cout:

This is known as out put stream in C and C++. Stream is a complicated thing, you will learn about it later. Think a stream as a door. The data is transferred through stream, **cout** takes data from computer and sends it to the output. For the moment it is a screen of the monitor. hence we use **cout** for output.

```
<<
```

The sign << indicates the direction of data. {insertion operators }{>>Extraction} Here it is towards **cout** and the function of

cout is to show data on the screen.

" Welcome to Virtual University of Pakistan"

The thing between the double quotes (" ") is known as character string. In C programming character strings are written in double quotes. Whatever is written after << and within quotation marks will be direct it to **cout**, **cout** will display it on the screen.

```
;
```

There is a semicolon (;) at the end of the above statement. This is very important. All C statements end with semicolon (;). Missing of a semicolon (;) at the end of statement is a **syntax error** and compiler will report an error during compilation. If there is only a semicolon (;) on a line than it will be called a null statement, i.e. it does nothing. The extra semicolons may be put at the end but are useless and aimless. Do not put semicolon (;) at a wrong place, it may cause a problem during the execution of the program or may cause a **logical error**.

In this program we give a fixed character string to **cout** and the program prints it to the screen as:

variable

During programming we need to store data. This data is stored in variables. Variables are locations in memory for storing data. The memory is divided into blocks. It can be viewed as pigeon-holes. You can also think of it as PO Boxes. In post offices there are different boxes and each has an address. Similarly in memory, there is a numerical address for each location of memory (block). It is difficult for us to handle these numerical addresses in our programs. So we give a name to these locations. These

names are variables. We call them variables because they can contain different values at different times.



The variable names in C may be started with a character or an underscore (_). But avoid starting a name with underscore (_). C has many libraries which contain variables and function names normally starting with underscore (_). So your variable name starting with underscore (_) may conflict with these variables or function names.

In a program every variable has

name

type

size

value

The variables having a name, type and size (type and size will be discussed later) are just empty boxes. They are useless until we put some value in them. To put some value in these boxes is known as assigning values to variables. In C language, we use assignment operator for this purpose.

Assignment Operator

In C language equal-to-sign (=) is used as assignment operator. Do not confuse the algebraic equal-to with the assignment operator. In Algebra $X = 2$ means the value of X is 2, whereas in C language $X = 2$ (where X is a variable name) means take the value 2 and put it in the memory location labeled as X , afterwards you can assign some other value to X , for example you can write $X = 10$, that means now the memory location X contains the value 10 and the previous value 2 is no more there. Assignment operator is a binary operator (a binary operator has two operands). It must have variable on left hand side and expression (that evaluates to a single value) on right hand side. This operator takes the value on right hand side and stores it to the location labeled as the variable on left hand side, e.g. $X = 5$, $X = 10 + 5$, and $X = X + 1$.

In C language the statement $X = X + 1$ means that add 1 to the value of X and then store the result in X variable. If the value of X is 10 then after the execution of this statement the value of X becomes 11. This is a common practice for incrementing the value of the variable by 'one' in C language. Similarly you can use the statement $X = X - 1$ for decrementing the value of the variable by one. The statement $X = X + 1$ in algebra is not valid except when X is infinity. So do not confuse assignment operator (=) with equal sign (=) in algebra. Remember that assignment operator must have a variable name on left hand side unlike algebra in which you can use expression on both sides of equal sign (=). For example, in algebra, $X + 5 = Y + 7$ is correct but incorrect in C language. The compiler will not

Data Types

understand it and will give error.

A variable must have a data type associated with it, for example it can have data types like integer, decimal numbers, characters etc. The variable of type Integer stores integer values and a character type variable stores character value. The primary difference between various data types is their size in memory. Different data types have different size in memory depending on the machine and compilers. These also affect the way they are displayed. The 'cout' knows how to display a digit and a character. There are few data types in C language. These data types are reserved words of C language. The reserve words can not be used as a variable name.

Let's take a look into different data types that the C language provides us to deal with whole numbers, real numbers and character data.

Whole Numbers



The C language provides three data types to handle whole numbers.

int
short
long

int Data Type

The data type int is used to store whole numbers (integers). The integer type has a space of 4 bytes (32 bits for windows operating system) in memory. And it is mentioned as 'int' which is a reserved word of C, so we can not use it as a variable name.

In programming before using any variable name we have to declare that variable with its data type. If we are using an integer variable named as 'i', we have to declare it as `int i;`

The above line is known as declaration statement. When we declare a variable in this way, it reserves some space in memory depending on the size of data type and labels it with the variable name. The declaration statement `int i;` reserves 4 bytes of memory and labels it as 'i'. This happens at the execution time.

Sample Program 1

```
#include <iostream.h>
main()
{
    int x; int
    y; int z; x
    = 5;
    y = 10;
    z = x + y;
    cout << "x = "; cout
    << x; cout << "
    y="; cout << y;
    cout << " z = x + y = "; cout
    << z;
}
```

The first three lines declare three variables x, y and z as following.

```
int x;
int y;
int z;
```

These three declarations can also be written on one line. C provides us the comma separator (,). The above three lines can be written in a single line as below

```
int x, y, z;
```

As we know that semicolon (;) indicates the end of the statement. So we can write many statements on a single line. In this way we can also write the above declarations in the following form

```
int x; int y; int z;
```

Read Yourselfes

For good programming practice, write a single statement on a single line.

Now we assign values to variables x and y by using assignment operator. The lines x = 5; and y = 10 assign the values 5 and 10 to the variables x and y, respectively. These statements put the values 5 and 10 to the memory locations labeled as x and y.

The next statement z = x + y; evaluates the expression on right hand side. It takes values stored in variables x and y (which are 5 and 10 respectively), adds them and by using the assignment operator (=), puts the value of the result, which is 15 in this case, to the memory location labeled as z.

Here a thing to be noted is that the values of x and y remains the same after this operation. In arithmetic operations the values of variables used in expression on the right hand side are not affected. They remain the same. But a statement like x = x + 1; is an exceptional case. In this case the value of x is changed.

The next line cout << "x = "; is simple it just displays 'x = ' on the screen.

Now we want to display the value of x after 'x = '. For this we write the statement cout << x;

Here comes the affect of data type on cout. The previous statement cout << "x = "; has a character string after << sign and cout simply displays the string. In the statement cout << x; there is a variable name x. Now cout will not display 'x' but the value of x. The cout interprets that x is a variable of integer type, it goes to the location x in the memory and takes its value and displays it in integer form, on the screen. The next line cout << "y = "; displays 'y = ' on the screen. And line cout << y; displays the value of y on the screen. Thus we see that when we write something in quotation marks it is displayed as it is but when we use a variable name it displays the value of the variable not name of the variable. The next two lines cout << "z = x + y = "; and cout << z; are written to display 'z = x + y = ' and the value of z that is 15.

Now when we execute the program after compiling, we get the following output:

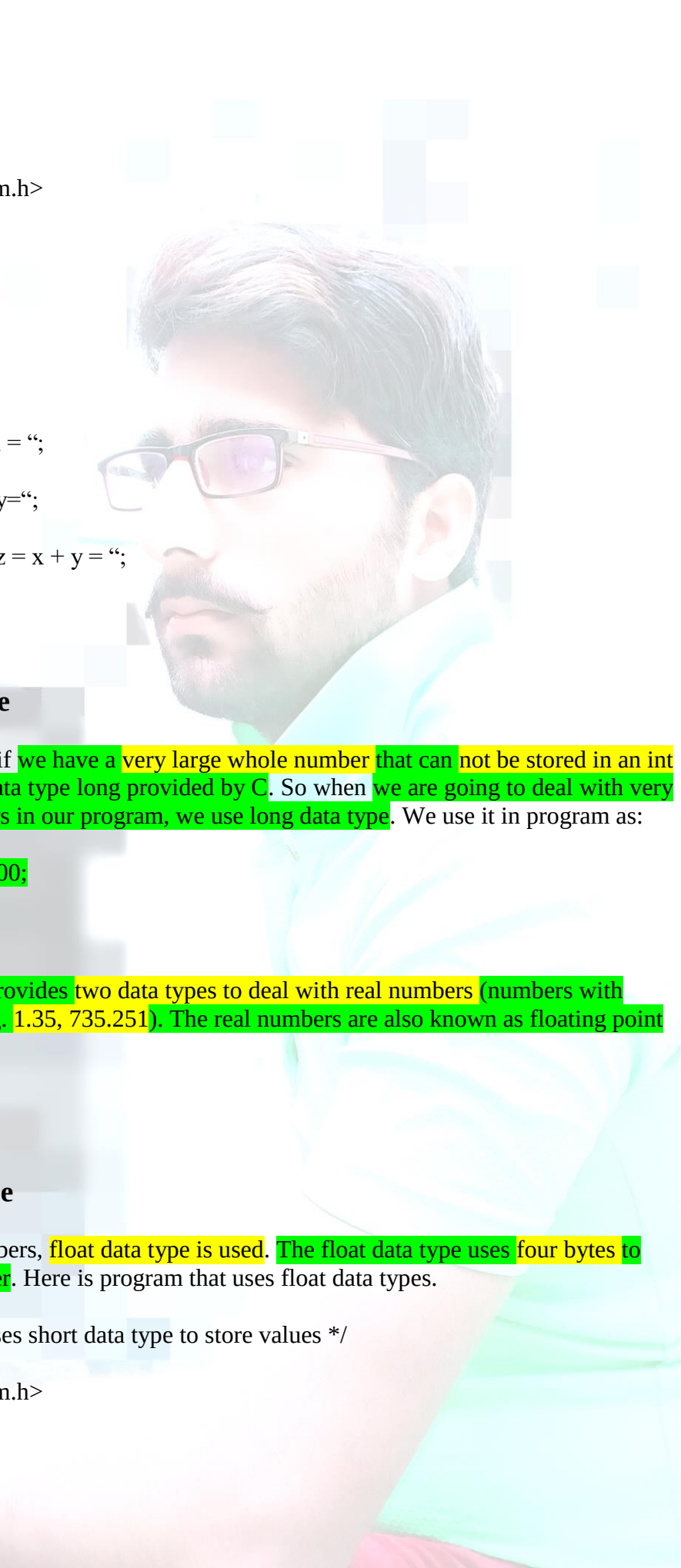
```
x = 5 y = 10 z = x + y = 15
```

short Data type

We noted that the integer occupies four bytes in memory. So if we have to store a small integer like 5, 10 or 20 four bytes would be used. The C provides another data type for storing small whole numbers which is called short. the size of short is two bytes and it can store numbers in range of -32768 to 32767. So if we are going to use a variable for which we know that it will not increase from 32767, for example the age of different people, then we use the data type short for age. We can write the

above sample program by using short instead of int.
integer type has a space of 4 bytes (32 bits for windows operating system) in memory.
/*This program uses short data type to store values */





```
#include <iostream.h>
main()
{
    short x;
    short y;
    short z;
    x = 5;
    y = 10;
    z = x + y;
    cout << "x = ";
    cout << x;
    cout << " y=";
    cout << y;
    cout << " z = x + y = ";
    cout << z;
}
```

long Data Type

On the other side if we have a very large whole number that can not be stored in an int then we use the data type long provided by C. So when we are going to deal with very big whole numbers in our program, we use long data type. We use it in program as:

```
long x = 300500200;
```

Real Numbers

The C language provides two data types to deal with real numbers (numbers with decimal points e.g. 1.35, 735.251). The real numbers are also known as floating point numbers.

```
float
double
```

float Data Type

To store real numbers, float data type is used. The float data type uses four bytes to store a real number. Here is program that uses float data types.

```
/*This program uses short data type to store values */
```

```
#include <iostream.h>
main()
{
    float x;
    float y;
    float z;
```

```

x = 12.35;
y = 25.57;
z = x + y;
cout << "x = ";
cout << x;
cout << "y = ";
cout << y;
cout << "z = x + y = ";
cout << z;
}

```

double Data Type

If we need to store a large real number which cannot be store in four bytes, then we use **double** data type. Normally the size of double is twice the size of float. In program we use it as:

```
double x = 345624.769123;
```

char Data Type

So far we have been looking on data types to store numbers, In programming we do need to store characters like **a,b,c** etc. For storing the character data C language provides **char** data type. By using char data type we can store characters in variables. While assigning a character value to a char type variable single quotes are used around the character as **'a'**.

```

/* This program uses short data type to store values */
#include <iostream.h>
main()
{
    char x;
    x = 'a';
    cout << "The character value in x = ";
    cout << x;
}

```

Arithmetic Operators

In C language we have the usual arithmetic operators for addition, subtraction, multiplication and division. C also provides a special arithmetic operator which is called modulus. All these operators are binary operators which means they operate on two operands. So we need two values for addition, subtraction, multiplication, division and modulus.

ARITHMETIC OPERATION	ARITHMETIC OPERATOR	ALGEBRAIC EXPRESSION	C EXPRESSION
Addition	+	$x + y$	$x + y$
Subtraction	-	$x - y$	$x - y$

Multiplication	*	xy	$x * y$
Division	/	$x \div y, x / y$	x / y
Modulus	%	$x \bmod y$	$x \% y$

Addition, subtraction and multiplication are same as we use in algebra.

There is one thing to note in division that when we use integer division (i.e. both operands are integers) yields an integer result. This means that if, for example, you are dividing 5 by 2 ($5 / 2$) it will give integer result as 2 instead of actual result 2.5. Thus in integer division the result is truncated to the whole number, the fractional part (after decimal) is ignored. If we want to get the correct result, then we should use float data type.

The modulus operator returns the remainder after division. This operator can only be used with integer operands. The expression $x \% y$ returns the remainder after x is divided by y . For example, the result of $5 \% 2$ will be 1, $23 \% 5$ will be 3 and $107 \% 10$ will be 7.

Precedence of Operators

The arithmetic operators in an expression are evaluated according to their precedence. The precedence means which operator will be evaluated first and which will be evaluated after that and so on. In an expression, the parentheses () are used to force the evaluation order. The operators in the parentheses () are evaluated first. If there are nested parentheses then the inner most is evaluated first.

The expressions are always evaluated from left to right. The operators $*$, $/$ and $\%$ have the highest precedence after parentheses. These operators are evaluated before $+$ and $-$ operators. Thus $+$ and $-$ operators has the lowest precedence. It means that if there are $*$ and $+$ operators in an expression then first the $*$ will be evaluated and then its result will be added to other operand. If there are $*$ and $/$ operators in an expression (both have the same precedence) then the operator which occurs first from left will be evaluated first and then the next, except you force any operator to evaluate by putting parentheses around it.

The following table explains the precedence of the arithmetic operators:

OPERATORS	OPERATIONS	PRECEDENCE (ORDER OF EVALUATION)
()	Parentheses	Evaluated first
$*$, $/$, or $\%$	Multiplication, Division, Modulus	Evaluated second. If there are several, they are evaluated from left to right
$+$ or $-$	Addition, Subtraction	Evaluated last. If there are several, they are evaluated from left to right

Lets look some examples.

What is the result of $10 + 10 * 5$?

The answer is 60 not 100. As $*$ has higher precedence than $+$ so $10 * 5$ is evaluated first and then the answer 50 is added to 10 and we get the result 60. The answer will be 100 if we force the addition operation to be done first by putting $10 + 10$ in parentheses. Thus the same expression rewritten as $(10 + 10) * 5$ will give the result 100. Note that how the parentheses affect the evaluation of an expression.

Similarly the expression $5 * 3 + 6 / 3$ gives the answer 17, and not 7. The evaluation of this expression can be clarified by writing it with the use of parentheses as $(5 * 3) + (6 / 3)$ which gives $15 + 2 = 17$. Thus you should be careful while writing arithmetic expressions.

TIP

Use spaces in the coding to make it easy to read and understand **Reserved words** can not be used as variable names

There is always a `main()` in a C program that is the starting point of execution Write one statement per line

Type parentheses '`()`' and braces '`{ }`' in pairs

Use parentheses for clarification in

arithmetic expressions **Don't forget**

semicolon at the end of each statement

C Language is case sensitive so variable names **`x` and `X`** are two different variables